

多項式・形式的べき級数 線形漸化的数列 C-recursive Fiduccia のアルゴリズム
Bostan-Mori のアルゴリズム 転置原理 FFT

Bostan-Mori のアルゴリズム (発展)

1. 概要

本講座では、Bostan-Mori のアルゴリズムに関する発展的な話題を扱います。

前回の講座 [線形漸化的数列の第 \$K\$ 項](#) では、線形漸化的数列の第 K 項を高速に求める方法として、**Fiduccia のアルゴリズム**と **Bostan-Mori のアルゴリズム**を解説しました。本講座ではこのうち Bostan-Mori のアルゴリズムについて、より発展的な話題を取り上げます。

本講座の前半では、次のような問題を統一的に解けることを確認します。

- 有理式表示された形式的べき級数の連続する係数 $A_L, A_{L+1}, \dots, A_R$ を求める。
- 数列を K 項だけずらした数列 $(A_K, A_{K+1}, A_{K+2}, \dots)$ の母関数を求める。
- 特性多項式 $\Gamma(x)$ による剰余 $x^K \bmod \Gamma(x)$ を求める。

本講座ではこれらの問題を、有理式表示された形式的べき級数の区間係数取得に帰着して解きます。この区間係数取得は、Bostan-Mori のアルゴリズムの **MSB-first 版**と呼ばれる変種として理解できます。また、このアルゴリズムが、第 K 項を求める通常の Bostan-Mori のアルゴリズムの**転置**と考えられることも確認します。

本講座の後半では、多項式乗算を **FFT** を用いて計算できる場合に、Bostan-Mori のアルゴリズムを高速化する方法を紹介します。具体的には、多項式の係数列ではなく、その **DFT 表現**を入出力に用いることで、効率よく計算を進められることを確認します。

多項式の DFT 表現をうまく扱うことでアルゴリズムを高速化する、という考え方は競技プログラミングでも様々な場面で活用できます。Bostan-Mori のアルゴリズムの高速化は、そのようなテクニックを学ぶ題材としても有用です。

2. 前提知識

次の講座の内容を理解していることを前提とします.

- 線形漸化的数列
- 線形漸化的数列の第 K 項

特に, 後者の講座における Bostan-Mori のアルゴリズムを理解していることが必要です.

第 5 節では, 離散フーリエ変換の定義や, 多項式乗算への利用方法を理解していることを前提とします. ただし, FFT の実装そのものには立ち入りません.

3. 区間係数取得

3.1. 共通の設定

定数係数線形漸化式

$$A_i = c_1 A_{i-1} + c_2 A_{i-2} + \cdots + c_d A_{i-d} \quad (i \geq d)$$

を満たす線形漸化的数列 A を考え, $A(x) = \sum_{i=0}^{\infty} A_i x^i$ をその母関数とします. また,

$$\begin{aligned} \Gamma(x) &= x^d - c_1 x^{d-1} - c_2 x^{d-2} - \cdots - c_d, \\ Q(x) &= 1 - c_1 x - c_2 x^2 - \cdots - c_d x^d \end{aligned}$$

とおきます. このとき母関数 $A(x)$ は, ある $d-1$ 次以下の多項式 $P(x)$ について $A(x) = \frac{P(x)}{Q(x)}$ が成り立つのでした.

本講座全体を通して, $d, A_i, c_i, A(x), \Gamma(x), P(x), Q(x)$ はここで定めたものを表すことにします.

単位的可換環 R を固定し, 数列の要素や多項式の係数はすべて R の元であるとし, また本講座で計算量について述べる際には, **算術計算量**, つまり環 R での演算の回数を表すこととします.

次数 d の多項式積の計算量を $M(d)$ と書きます.

3.2. 区間係数の記号

$$f(x) = \sum_{i=0}^{\infty} f_i x^i$$

を形式的べき級数とします. この x^i の係数を

$$[x^i]f(x)$$

で表します. ただし本講座では, i は非負整数に限定せず, 負の整数の場合にもこの記号を用います. $i < 0$ の場合 $[x^i]f(x) = 0$ であると約束します. また, 非負整数 L, R が $L \leq R$ を満たすとき,

$$[x^{[L,R]}]f(x) = \sum_{i=L}^R f_i x^{i-L}$$

と定義します. 例えば,

$$\begin{aligned} [x^{[2,4]}](x + 2x^2 + 3x^3 + 4x^4 + 5x^5) &= 2 + 3x + 4x^2, \\ [x^{[-2,3]}](x + 2x^2 + 3x^3 + 4x^4 + 5x^5) &= x^3 + 2x^4 + 3x^5 \end{aligned}$$

です.

3.3. 区間係数取得の問題

本節では次の問題を扱います.

【問題 1】

$d-1$ 次以下の多項式 $P(x)$, d 次以下の多項式 $Q(x)$ (ただし $[x^0]Q(x) = 1$) および, $L \leq R$ を満たす非負整数 L, R が与えられるので,

$$[x^{[L,R]}] \frac{P(x)}{Q(x)}$$

を求めてください.

$P(x)$ が $d-1$ 次以下の多項式であるとき,

$$[x^{[L,R]}] \frac{P(x)}{Q(x)} = [x^{[L,R]}] \left(P(x) \cdot \frac{1}{Q(x)} \right)$$

に寄与するのは, $\frac{1}{Q(x)}$ のうちで $L - d + 1$ 次以上 R 次以下の部分に限られます. したがって,

$$[x^{[L-d+1,R]}] \frac{1}{Q(x)}$$

を求められれば, **【問題 1】** を解くことができます. したがって以下ではこの問題の代わりに次の問題の解法を説明します.

【問題 2】

d 次以下の多項式 $Q(x)$ (ただし $[x^0]Q(x) = 1$) および, $L \leq R$ を満たす整数 L, R (ただし $R \geq 0$) が与えられるので,

$$[x^{[L,R]}] \frac{1}{Q(x)}$$

を求めてください.

なお本講座では記述の場合分けを減らすために, **【問題 2】** では L は (非負整数ではなく) 整数として定式化しています.

3.4. 区間係数取得のアルゴリズム

【問題 2】 を解くアルゴリズムを解説します.

$Q(x)$ を d 次以下の多項式とすると, $Q(x)Q(-x)$ の奇数次部分は 0 であり, ある d 次以下の多項式 $V(x)$ が存在して

$$Q(x)Q(-x) = V(x^2)$$

が成り立つのでした. Bostan-Mori のアルゴリズムと同様にこの関係を用いることで, R がより小さい同じ形の問題に帰着します. まず,

$$\begin{aligned} [x^{[L,R]}] \frac{1}{Q(x)} &= [x^{[L,R]}] \frac{Q(-x)}{Q(x)Q(-x)} \\ &= [x^{[L,R]}] Q(-x) \frac{1}{V(x^2)} \end{aligned}$$

と変形します。 $Q(-x)$ は d 次以下の多項式なので、

$$S(x) = [x^{[L-d,R]}] \frac{1}{V(x^2)}$$

とおけば、

$$[x^{[L,R]}] \frac{1}{Q(x)} = [x^{[d,R-L+d]}] Q(-x) S(x)$$

が成り立ちます。したがって、 $S(x)$ が求められれば、求めたい係数区間を復元できます。

次に、 $S(x)$ を求める方法を考えます。 $\frac{1}{V(x^2)}$ は偶数次の係数だけを持つ形式的べき級数です。また、整数 i に対して

$$L - d \leq 2i \leq R \iff \left\lceil \frac{L-d}{2} \right\rceil \leq i \leq \left\lfloor \frac{R}{2} \right\rfloor$$

が成り立ちます。したがって、

$$L' = \left\lceil \frac{L-d}{2} \right\rceil, \quad R' = \left\lfloor \frac{R}{2} \right\rfloor$$

とし、

$$W(x) = [x^{[L',R']}] \frac{1}{V(x)}$$

とすれば、 $S(x)$ は $W(x)$ から復元できます。具体的には、

$$S(x) = x^{2L'-L+d} W(x^2)$$

となります。これにより、

$$[x^{[L,R]}] \frac{1}{Q(x)}$$

を求める問題が、

$$[x^{[L', R']}] \frac{1}{V(x)}$$

を求める問題に帰着できました。つまり、問題の形を保ったまま、 R を $R' = \lfloor \frac{R}{2} \rfloor$ にすることができました。

この操作を繰り返すと、最終的には $R = 0$ の場合に帰着されます。 $R = 0$ の場合 $[x^{[L, 0]}] \frac{1}{Q(x)}$ の答えは明らかに x^{-L} です。

まとめると、以下の擬似コードにより **【問題 2】** を解くことができます。

```
function range_coefficients_of_inverse(Q(x), d, L, R):
    # Q(x) is a polynomial of degree at most d and Q(0)=1
    # returns [x^{[L,R]}] 1/Q(x)

    if R==0:
        return x^{-L}

    L' := ceil((L - d) / 2)
    R' := floor(R / 2)

    V(x) := even part of Q(x)Q(-x)
    W(x) := range_coefficients_of_inverse(V(x), d, L', R')

    S(x) := x^{2L' - L + d} W(x^2)
    return [x^{[d, R-L+d]}] Q(-x)S(x)
```

▶ middle product について

3.5. 計算量について

k 回目の再帰呼び出しにおける $Q(x), S(x), L, R$ を $Q_k(x), S_k(x), L_k, R_k$ と書くことにします。

イテレーションごとに多項式乗算 $Q_k(x)Q_k(-x)$, $Q_k(-x)S_k(x)$ を計算します。このうち $Q_k(x)$ は常に d 次以下です。一方、 $S_k(x)$ の大きさ $R_k - L_k + d + 1$ はアルゴリズムの進行によって変化します。

$m_k = (R_k - L_k + 1) - d$ とすれば, m_{k+1} は $m_k/2$ 程度になります. つまり区間長と d の差が半分になっていきます. このことから計算量を評価すると,

$$O(M(d) \log R + M(R - L + d))$$

あるいは次数の大きく違う多項式積についてより慎重な実装をすれば

$$O(M(d) \log R + \frac{R - L + 1 + d}{d} M(d))$$

であることが分かります.

3.6. 参考：転置原理による解釈

【問題 2】 の解を

$$[x^{[L,R]}] \frac{1}{Q(x)} = \sum_{i=0}^{R-L} a_i x^i$$

として,

$B(x) = \sum_{i=0}^{R-L} b_i x^i$ を $R - L$ 次以下の多項式とすると,

$$[x^R] \frac{B(x)}{Q(x)} = \sum_{i=0}^{R-L} a_i b_{R-L-i}$$

が成り立ちます. したがって,

- $[x^{[L,R]}] \frac{1}{Q(x)}$ を出力する問題.
- $B(x)$ を入力として $[x^R] \frac{B(x)}{Q(x)}$ を出力する問題.

は, 係数列の並び順を反転するなどの差を無視すれば, 互いに転置問題の関係にあることが分かります.

転置原理より, これらの問題は同程度の算術計算量で解くことができ, 一方のアルゴリズムから他方のアルゴリズムを導出することも可能です. 特に, $[x^R] \frac{B(x)}{Q(x)}$ を求めるアルゴリズムの転置として, $[x^{[L,R]}] \frac{1}{Q(x)}$ を求めるアルゴリズムを導出することも可能です.

$[x^R] \frac{B(x)}{Q(x)}$ を求めるアルゴリズムは、線形漸化的数列の第 K 項 において Bostan-Mori のアルゴリズムとして解説しました。ただし、 $B(x)$ の次数が d 以上の場合には少し実装の修正が必要な場合もあるので注意してください。そのような修正を行った上で転置原理を適用すれば、機械的に区間係数を求めるアルゴリズムを導出することもできます。

また同じように、**【問題 1】** つまり $\frac{P(x)}{Q(x)}$ の区間係数を求める問題は、 $B(x)$ を入力として $[x^R] \frac{P(x)B(x)}{Q(x)}$ を求める問題の転置として解くこともできます。

3.7. LSB-first と MSB-first

Bostan-Mori のアルゴリズムが発表された論文 (文献 1) では、3.4 節で解説したアルゴリズム (の特に $[L, R] = [K - d + 1, K]$ の場合) を **Bostan-Mori の MSB-first アルゴリズム** として扱っています。ここでは MSB-first というものの意味について簡単に解説します。

Bostan-Mori のアルゴリズムの中核をなすのは、次の式変形です：

$$\frac{1}{Q(x)} = \frac{Q(-x)}{Q(x)Q(-x)} = Q(-x) \cdot \frac{1}{V(x^2)}$$

この式変形を反復することは、

$$\frac{1}{Q(x)} = Q_0(x) Q_1(x^2) Q_2(x^4) Q_3(x^8) \cdots = \prod_{k=0}^{\infty} Q_k(x^{2^k})$$

という無限積表示を得ることに相当します。通常の Bostan-Mori のアルゴリズムはこの無限積を、 k について昇順に処理していくことで答えを求めるものと解釈することができます。一方 3.4 節のアルゴリズムでは、 k について降順に処理していくことで答えを求めるものと解釈することができます。

このような観点から、通常の Bostan-Mori のアルゴリズムを **LSB-first**、本節で扱ったアルゴリズムを **MSB-first** と呼ぶ場合があります。

3.8. 参考：形式的べき級数逆元

【問題 2】 は明らかに、形式的べき級数逆元の計算を含む問題設定です。つまり、次の問題を含みます。

【問題 3】

$[x^{[0,d]}] \frac{1}{Q(x)}$ を求めよ。

この計算は $O(d \log d)$ で行えることが知られており、特に Newton 法を用いる方法が一般的だと思います。

一方で 3.4 節のアルゴリズムを以下の点に注意して修正すれば、同様の計算量を達成することができます。

- $L < 0$ の場合の区間係数は、 $[0, R]$ での区間係数に帰着できる。
- $Q(x)$ の係数のうち、 $R + 1$ 次以上の部分は無視する。

これらの修正を行えば、イテレーションのたびに入出力の多項式の大きさを半分程度にすることができ、 $O(d \log d)$ 時間で答えを求めることができます。

このように $Q(x)Q(-x)$ を用いて形式的べき級数逆元を再帰的に計算する方法は、Schönhage による先行研究があります（[文献 7](#)）。

3.9. 実装例

[Library Checker "Consecutive Terms of Linear Recurrent Sequence"](#) への提出です。

- <https://judge.yosupo.jp/submission/376868>
- <https://judge.yosupo.jp/submission/376905> (middle product による高速化を行ったもの)

[Library Checker "Inv of Formal Power Series"](#) への提出です。

- <https://judge.yosupo.jp/submission/376903>

4. 区間係数取得の応用

第 3 節で扱った問題と関連して理解できる問題を挙げます。

4.1. 線形漸化的数列のシフト

【問題 4】

$P(x)$, $Q(x)$ および非負整数 K が与えられるので, 数列 $(A_K, A_{K+1}, A_{K+2}, \dots)$ の母関数の有理式表示を求めてください.

$B_i = A_{K+i}$ により数列 B を定め, その母関数を $B(x)$ とおきます.

まず, $A(x) = \frac{P(x)}{Q(x)}$ の係数列は線形漸化的数列で, $Q(x)$ に対応する定数係数線形漸化式を満たします. 明らかに $B = (A_K, A_{K+1}, A_{K+2}, \dots)$ も同じ定数係数線形漸化式を満たすため, その母関数はある $d-1$ 次以下の多項式 $S(x)$ を用いて

$$B(x) = \sum_{i=0}^{\infty} A_{K+i} x^i = \frac{S(x)}{Q(x)}$$

と表すことができます. これを求めるためには, まず

$$[x^{[0, d-1]}] B(x) = [x^{[K, K+d-1]}] A(x) = [x^{[K, K+d-1]}] \frac{P(x)}{Q(x)}$$

を求めます. この多項式に $Q(x)$ をかけて, $d-1$ 次以下の部分を取り出したものが $S(x)$ となります.

したがってこの問題は $O(M(d) \log K)$ 時間で解くことができます.

4.2. x^K の多項式剰余

【問題 5】

非負整数 K が与えられるので, $x^K \bmod \Gamma(x)$ を求めてください.

この問題は繰り返し二乗法でも解くことができます（Fiduccia のアルゴリズム）が、Bostan-Mori のアルゴリズムによる区間係数取得を利用しても解くことができます。

まず

$$U(x) = [x^{[K-d+1, K]}] \frac{1}{Q(x)}$$

とします。また

$$R(x) = \sum_{i=0}^{d-1} r_i x^i = x^K \bmod \Gamma(x)$$

とし、その係数列を逆順にした多項式を

$$\tilde{R}(x) = \sum_{i=0}^{d-1} r_{d-1-i} x^i$$

と書くことにします。 $U(x)$, $\tilde{R}(x)$ はともに、 A_K を求める問題と関係があります。

線形漸化的数列の第 K 項 **【定理 3】** で解説したように、

$$A_K = \sum_{i=0}^{d-1} r_i A_i$$

が成り立ちます。したがって

$$A_K = [x^{d-1}] A(x) \tilde{R}(x)$$

と表すことができます。次に、

$$A_K = [x^K] \frac{P(x)}{Q(x)} = [x^K] P(x) \cdot \frac{1}{Q(x)}$$

であることに注目すると $P(x)$ が $d-1$ 次以下の多項式であることから、

$$A_K = [x^{d-1}] P(x) U(x)$$

と表すことができます。したがって

$$A_K = [x^{d-1}] A(x) \tilde{R}(x) = [x^{d-1}] P(x) U(x)$$

となります。さらに $P(x) = A(x)Q(x)$ から

$$[x^{d-1}]A(x)\tilde{R}(x) = [x^{d-1}]A(x)Q(x)U(x)$$

が得られます。ここで、 $\tilde{R}$, $Q(x)$, $U(x)$ はいずれも $A(x)$ に依らずに定まる（言い換えれば、 $Q(x)$ と K だけから定まる）多項式です。この等式が任意の初期値 $A_0, A_1, \dots, A_{d-1}$ に対して成り立つため、

$$\tilde{R}(x) = Q(x)U(x) \bmod x^d$$

であることが分かります。より厳密には、初期値を $A(x) \equiv x^{d-1-i} \pmod{x^d}$ となるように選んだ場合から

$$[x^i]\tilde{R}(x) = [x^i]Q(x)U(x)$$

であることが分かります。以上により、 $R(x) = x^K \bmod \Gamma(x)$ を求めるには、

- $\tilde{R}(x) = Q(x)U(x) \bmod x^d$ を求める。
- その係数列を逆順にする。

とすればよいです。計算量は $O(M(d) \log K)$ となります。

4.3. 参考：Fiduccia のアルゴリズムの利用

【問題 5】 は、繰り返し二乗法によって $O(M(d) \log K)$ 時間で解くこともでき、この計算は Fiduccia のアルゴリズムの主要な部分でした。

本講座では、【問題 2】 の解を元にして 【問題 5】 の解を導きましたが、逆に 【問題 5】 の解を元にして、【問題 2】、【問題 1】、【問題 4】 の解を得ることも可能です。したがって、本講座で扱っている問題は、すべて Fiduccia のアルゴリズムを元にして同じ漸近計算量で解くこともできます。

ただし、両アルゴリズムを適切に実装した場合には、漸近計算量は同じであっても、Bostan-Mori のアルゴリズムに基づく方法の方が、算術計算量の定数倍の観点で有利になることが知られています。

5. FFT を用いる場合の定数倍高速化

5.1. DFT, FFT の利用

これまでは、数列の項 A_i や漸化式・多項式の係数 c_i が環 R の元であるとして議論してきました。第 5 節ではさらに、次の状況を前提とします。

- $n = 2^k$ を、 $d + 1$ 以上の最小の 2 冪とする。
- R が体であり、1 の原始 $2n$ 乗根 ζ_{2n} が存在する。

競技プログラミングにおいては、次のどちらかの状況を考えれば十分でしょう。

- $R = \mathbb{C}$ である。このとき $\zeta_{2n} = \exp(2\pi i/2n)$ ととることができる。
- $R = \mathbb{F}_p$ (p は素数) であり、 $p \equiv 1 \pmod{2n}$ を満たす。 r を p を法とする原始根のひとつとすれば、 $\zeta_{2n} = r^{(p-1)/2n} \bmod p$ ととることができる。
- 例えば $p = 998244353$ かつ $d \leq 2^{22} - 1$ とした場合にこの条件が成り立つ。

1 の原始 $2n$ 乗根 ζ_{2n} をひとつとり固定します。このとき ζ_{2n}^2 は 1 の原始 n 乗根となります。そこで、

$$\zeta_n = \zeta_{2n}^2$$

と書きます。また、本節を通して ζ_{2n} を ζ と略記します。

本節では、長さ n の **DFT (離散フーリエ変換)** を $\mathcal{F}_n$ と書きます。つまり、 $n - 1$ 次以下の多項式 $P(x)$ に対して

$$\mathcal{F}_n[P(x)]_i = P(\zeta_n^i) \quad (0 \leq i < n)$$

と書くことにします。この列 $\mathcal{F}_n[P(x)]$ を、 $P(x)$ の長さ n の **DFT 表現** と呼ぶことにします。長さ $2n$ の場合の $\mathcal{F}_{2n}$ も同様に定義します。

DFT およびその逆変換 IDFT は、**FFT アルゴリズム** を利用することで、算術計算量

$$O(n \log n) = O(d \log d)$$

で求めることができるのでした。また、 $P(x), Q(x)$ が d 次以下の多項式であるとき、

$$\mathcal{F}_{2n}[P(x)Q(x)]_i = \mathcal{F}_{2n}[P(x)]_i \cdot \mathcal{F}_{2n}[Q(x)]_i \quad (0 \leq i < 2n)$$

が成り立ちます。このことから、多項式積 $P(x)Q(x)$ は、

- $\mathcal{F}_{2n}[P(x)]$ を求める.
- $\mathcal{F}_{2n}[Q(x)]$ を求める.
- 上で得た 2 つの列の成分ごとの積を求め, それを IDFT する.

という手順によって $O(d \log d)$ 時間で計算できます.

▶ 注意 1

▶ 注意 2

5.2. 高速化の概要

Bostan-Mori のアルゴリズムにおける各イテレーションでは, 次の計算を行います.

```
function Bostan_Mori_step(P(x), Q(x), K):
    U(x) := P(x)Q(-x)
    V(x) := Q(x)Q(-x)

    if K is even:
        P'(x) := even part of U(x)
    else:
        P'(x) := odd part of U(x)
    Q'(x) := even part of V(x)

    return P'(x), Q'(x)
```

この計算には 2 つの多項式乗算 $P(x)Q(-x)$, $Q(x)Q(-x)$ が含まれるため, 一見すると算術計算量は $2M(d)$ 程度です.

本節では, 多項式を係数列ではなく, DFT 表現のまま持って計算することで, この算術計算量を改善します. つまり, 次の擬似コードと等価な処理を, 係数列に完全には戻さずに実行することが目標です.

擬似コードでは, $\mathcal{F}_n[P(x)]$ を $\mathbf{F}_n(\mathbf{P})$, $\mathcal{F}_{2n}[P(x)]$ を $\mathbf{F}_{2n}(\mathbf{P})$ と表しています.

```

function Bostan_Mori_step_DFT(F_2n(P), F_2n(Q), K):
  # input: F_2n(P), F_2n(Q)
  # output: F_2n(P'), F_2n(Q')

  U(x) := P(x)Q(-x)
  V(x) := Q(x)Q(-x)

  if K is even:
    P'(x) := even part of U(x)
  else:
    P'(x) := odd part of U(x)
  Q'(x) := even part of V(x)

  return F_2n(P'), F_2n(Q')

```

以降では、この入出力を DFT 表現のまま実現するために必要な操作を順に説明します。

5.3. $Q(-x)$ の長さ $2n$ の DFT 表現

まず、 $Q(-x)$ の DFT 表現を考えます。 ζ を原始 $2n$ 乗根とします。 このとき、

$$\mathcal{F}_{2n}[Q(x)]_i = Q(\zeta^i)$$

です。 一方、 $Q(-x)$ の DFT 表現は

$$\mathcal{F}_{2n}[Q(-x)]_i = Q(-\zeta^i) = Q(\zeta^{n+i})$$

です。 したがって、

$$\mathcal{F}_{2n}[Q(-x)]_i = Q(\zeta^{i+n}) = \mathcal{F}_{2n}[Q(x)]_{i+n}$$

です。 ただし添字は $2n$ を法として考えます。

つまり、 $Q(-x)$ の長さ $2n$ の DFT 表現は、 $Q(x)$ の長さ $2n$ の DFT 表現の添字を n だけずらすことで得られます。

5.4. 偶数次部分・奇数次部分の長さ n の DFT 表現

次に、 $2n - 1$ 次以下の多項式 $G(x)$ の長さ $2n$ の DFT 表現から、偶数次部分 $G_e(x)$ 、奇数次部分 $G_o(x)$ の長さ n の DFT 表現を求めることを考えます。

$$G(x) = G_e(x^2) + xG_o(x^2)$$

でした。したがって

$$\begin{aligned} G(\zeta^i) &= G_e(\zeta^{2i}) + \zeta^i G_o(\zeta^{2i}), \\ G(\zeta^{i+n}) &= G(-\zeta^i) = G_e(\zeta^{2i}) - \zeta^i G_o(\zeta^{2i}) \end{aligned}$$

が成り立ちます。したがって、これらを足し引きすることで、

$$\begin{aligned} G_e(\zeta^{2i}) &= \frac{G(\zeta^i) + G(\zeta^{i+n})}{2}, \\ G_o(\zeta^{2i}) &= \frac{G(\zeta^i) - G(\zeta^{i+n})}{2\zeta^i} \end{aligned}$$

が得られます。つまり DFT 表現について

$$\begin{aligned} \mathcal{F}_n[G_e(x)]_i &= \frac{\mathcal{F}_{2n}[G(x)]_i + \mathcal{F}_{2n}[G(x)]_{i+n}}{2}, \\ \mathcal{F}_n[G_o(x)]_i &= \frac{\mathcal{F}_{2n}[G(x)]_i - \mathcal{F}_{2n}[G(x)]_{i+n}}{2\zeta^i} \end{aligned}$$

が成り立ちます。したがって、 $G(x)$ の長さ $2n$ の DFT 表現から、 $G_e(x)$ 、 $G_o(x)$ の長さ n の DFT 表現を $O(n)$ 時間で求めることができます。

5.5. 長さ n から長さ $2n$ へ

$n - 1$ 次以下の多項式 $G(x)$ の長さ n の DFT 表現 $\mathcal{F}_n[G(x)]$ が求まっているとします。 $\zeta_n^i = \zeta_{2n}^{2i}$ なので、これは $\mathcal{F}_{2n}[G(x)]$ の偶数番目の値がすべて求まっていることに相当します。

この場合に長さ $2n$ の DFT 表現 $\mathcal{F}_{2n}[G(x)]$ を得るには、奇数番目の値

$$G(\zeta_{2n}^{2i+1}) \quad (0 \leq i < n)$$

を求めればよいです。 $H(x) = G(\zeta x)$ とすれば、求めるべきは

$$H(\zeta_n^i) \quad (0 \leq i < n)$$

つまり、 $H(x)$ の長さ n の DFT 表現となります。これを求めるには、

- $\mathcal{F}_n[G(x)]$ に対して長さ n の IDFT を行い、 $G(x)$ を求める。
- $H(x) = G(\zeta x)$ を求める。
- $H(x)$ の長さ n での DFT 表現 $\mathcal{F}_n[H(x)]$ を求める。

とすればよいです。つまり、長さ n の IDFT と、DFT を 1 回ずつ行えばよいです。

5.6. まとめ

以上の操作を用いると、Bostan-Mori の 1 イテレーションは次のように DFT 表現のまま実行できます。

```
function Bostan_Mori_step_DFT(F_2n(P), F_2n(Q), K):
  # input: F_2n(P), F_2n(Q)
  # output: F_2n(P'), F_2n(Q')

  calculate F_2n(Q_minus) # Q_minus(x) = Q(-x), by shifting indices

  F_2n(U) := pointwise product of F_2n(P) and F_2n(Q_minus)
  F_2n(V) := pointwise product of F_2n(Q) and F_2n(Q_minus)

  if K is even:
    calculate F_n(P') # P'(x) = even part of U(x)
  else:
    calculate F_n(P') # P'(x) = odd part of U(x)

  calculate F_n(Q') # Q'(x) = even part of V(x)

  calculate F_2n(P') # from F_n(P')
  calculate F_2n(Q') # from F_n(Q')

  return F_2n(P'), F_2n(Q')
```

これらの計算のうち、 $O(n)$ 時間で行えない部分は、`calculate F\_2n(P')`、`calculate F\_2n(Q')` の部分のみです。これらは 5.5 節で説明した計算です。 P' 、 Q' それぞれについて長さ n の IDFT と DFT を 1 回ずつ用いるので、ステップ全体では長さ n の DFT, IDFT が

合計 4 回必要です。これは、長さ $2n$ の DFT, IDFT に換算するとおおよそ 2 回分に相当します。

一方、通常の変数乗算を FFT で行う場合、長さ $2n$ の DFT, IDFT が 3 回程度必要です。したがって、Bostan-Mori の 1 イテレーションの算術計算量を、

$$\frac{2}{3}M(d)$$

程度にすることができました。これを各イテレーションで行うことで、 $[x^K] \frac{P(x)}{Q(x)}$ を求める通常の Bostan-Mori のアルゴリズムの算術計算量は、

$$\frac{2}{3}M(d) \log K$$

程度になり、単純にイテレーションごとに多項式乗算を 2 回行うよりも計算量が改善されます。

5.7. 実装例

Library Checker "Kth term of Linearly Recurrent Sequence" への提出です。

- <https://judge.yosupo.jp/submission/376901>

DFT, IDFT には AtCoder Library の `butterfly`, `butterfly_inv` を用いています。これらは DFT の結果を **bit reverse 順** に並べた列として扱います。本講座の説明では通常の添字順で DFT 表現を書いているため、実装を読む際には添字の対応に注意してください。

5.8. 参考：区間係数取得の高速化

本講座の前半で扱った 【問題 2】 等についても類似の手法によって、高速化が可能です。

アルゴリズムの入出力となる多項式を DFT 表現に置き換えるという考え方や、本節で得たアルゴリズムを転置するという考え方により、高速なアルゴリズムを導出することができます。

6. 関連問題

- https://judge.yosupo.jp/problem/consecutive_terms_of_linear_recurrent_sequence

- https://judge.yosupo.jp/problem/inv_of_formal_power_series
- https://judge.yosupo.jp/problem/kth_term_of_linearly_recurrent_sequence
- https://atcoder.jp/contests/pakencamp-2022-day3/tasks/pakencamp_2022_day3_k

7. まとめ

本講座では、Bostan-Mori のアルゴリズムに関する発展的な話題を扱いました。

前半では、有理式表示された母関数の区間係数

$$[x^{[L,R]}] \frac{P(x)}{Q(x)}$$

を求める問題を考えました。この問題を解くことで、さらに線形漸化的数列をシフトした場合の母関数の有理式表示や、 $x^K \bmod \Gamma(x)$ の計算を行えることを確認しました。また、この問題を解くアルゴリズムは、通常の Bostan-Mori のアルゴリズムの転置と見ることができます。

後半では、FFT が利用可能な場合に、Bostan-Mori のアルゴリズムを定数倍高速化できることを説明しました。Bostan-Mori のアルゴリズムの各イテレーションでは、通常の見方では、

$$P(x)Q(-x), \quad Q(x)Q(-x)$$

という 2 回の多項式乗算を行うことになります。しかし、DFT 表現のまま計算を進め、 $Q(-x)$ の DFT 表現や、偶数次部分・奇数次部分の DFT 表現を効率よく求めることで、2 回の多項式乗算よりも高速に同等の計算を行うことができます。

本講座で扱ったように、Bostan-Mori のアルゴリズムは、線形漸化的数列の第 K 項計算に限らず、係数区間の取得、母関数のシフト、特性多項式による剰余計算などにも応用できます。また、Bostan-Mori のアルゴリズムやその転置は、形式的べき級数合成や power projection などのアルゴリズムを理解する上でも重要です。

8. 参考文献

1. Alin Bostan, Ryuhei Mori. A Simple and Fast Algorithm for Computing the N -th Term of a Linearly Recurrent Sequence. Proceedings of Symposium on

Simplicity in Algorithms (SOSA). pp. 118–132. 2021. DOI: 10.1137/1.9781611976496.14. <https://doi.org/10.1137/1.9781611976496.14>. (Preprint). <https://arxiv.org/abs/2008.08822>.

2. Ryuhei Mori. 線形漸化的数列のN項目の計算. Qiita. <https://qiita.com/ryuhe1/items/da5acbcce4ac1911f47a>. (閲覧日: 2026-06-04).
3. Ryuhei Mori. $X^N \bmod P(X)$ の計算. Qiita. <https://qiita.com/ryuhe1/items/c18ddbb834eed724a42b>. (閲覧日: 2026-06-04).
4. noshi91のメモ. メモ: Bostan-Mori 法で計算できるものまとめ. <https://noshi91.hatenablog.com/entry/2023/03/29/215553>. (閲覧日: 2026-06-04).
5. noshi91のメモ. FFT の回数を削減するテクニック集. <https://noshi91.hatenablog.com/entry/2023/12/10/163348>. (閲覧日: 2026-06-04).
6. OI Wiki. 常系数斉次線性递推. <https://oi-wiki.org/math/poly/linear-recurrence/>. (閲覧日: 2026-06-04).
7. Arnold Schönhage. Variations on computing reciprocals of power series. Information Processing Letters. Vol. 74. No. 1–2. pp. 41–46. 2000. DOI: 10.1016/S0020-0190(00)00044-2. [https://doi.org/10.1016/S0020-0190\(00\)00044-2](https://doi.org/10.1016/S0020-0190(00)00044-2).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)